CLI Coding Assistant

Project Report — A command-line coding assistant built from scratch in Python, using Google's Gemini API, implementing agentic tool-calling for reading, creating, editing, and deleting files with human-in-the-loop safety controls.

Author	Shaurya Pathak
Language / Stack	Python 3.13, Google Gemini API (free tier), google-genai SDK, python-dotenv
Version control	Git + GitHub
Project status	Complete (v1) — Steps 1 through 5

Abstract

This report documents the design and implementation of a small-scale, from-scratch clone of the core mechanics behind agentic CLI coding tools such as Claude Code. The project was built incrementally in five stages: a stateful chat loop, a file-reading tool exposed via LLM function calling, a precision text-replacement editing tool with diff previews, file creation and deletion tools with tiered safety confirmations, and a final hardening pass covering edge-case and interrupt handling. Every safety mechanism described was manually tested against both its intended (accept) and rejection paths, including adversarial cases such as ambiguous text matches and mid-prompt keyboard interrupts.

Contents

1. Objective and Scope

The goal of this project was to build a CLI-based coding assistant that can read a local codebase, propose edits, and apply them only with the user's confirmation — a minimal, educational analogue of tools such as Claude Code. The project was deliberately scoped to five core capabilities:

Conversational loop — Accept input, send it to an LLM, print the response, and retain conversation history across turns.

File I/O — Allow the model to read real files from disk so it can reason about actual code rather than guessing.

Edit proposals — Give the model a structured, unambiguous way to propose a specific change to a file.

Diff display — Render a human-readable preview of any proposed change before it is applied.

Safety gate — Require explicit human confirmation before any write, create, or delete operation reaches disk.

The project intentionally avoided over-scoping: rather than aiming for full IDE-like functionality, it focused on correctly implementing the underlying mechanics — tool calling, state, and safety — that such tools depend on.

2. System Architecture

The final codebase is organized into three modules, following a separation-of-concerns design so that each file has a single, clear responsibility:

Module	Responsibility
tools.py	All file-system operations (read, edit, create, delete) and their LLM-facing schemas; owns all confirmation-prom
api.py	Gemini client setup, model selection, and network resilience (retry logic for transient and quota errors).
assistant.py	The interactive loop: accepts input, dispatches tool calls via a lookup table, and prints the model's final respons

New tools are added by writing a function and schema in tools.py and registering both in two lookup structures (TOOL_FUNCTIONS and ALL_TOOL_SCHEMAS); assistant.py requires no changes to support new tools, since it dispatches purely by name lookup.

2.1 Control flow

1. User enters a message at the CLI prompt.
2. The message is sent to the Gemini chat session, along with the full set of registered tool schemas.
3. The model's response is inspected: if it contains a function call, the corresponding local Python function is executed and its result is sent back to the model as a new message.
4. This exchange repeats in a loop for as many tool calls as the model requires.

5. Once the model returns plain text (no further function call), that text is printed to the user as the final answer.

A central architectural principle underlies this entire design: the model can only *request* an action via a function call; it never executes anything directly. All actual file-system access is performed by this program's own code, which is therefore the sole point of control for every safety mechanism in the system.

3. Environment and Tooling Setup

3.1 Isolation: virtual environment

A Python virtual environment (venv) was used to isolate project dependencies from the system Python installation, preventing version conflicts with other projects. During development, a conflict was identified between an active Conda base environment and the project venv; this was resolved by explicitly verifying the active interpreter with `which python` before installing packages.

3.2 Secrets management

The Gemini API key is stored in a local `.env` file and loaded into the process environment at runtime via `python-dotenv`, rather than being hardcoded into source. The key is loaded before the API client is constructed, since client initialization is the point at which the SDK reads the credential from the environment.

```
from dotenv import load_dotenv
load_dotenv()

from google import genai
client = genai.Client()  # reads GEMINI_API_KEY from the environment
```

3.3 Version control hygiene

A `.gitignore` file excludes the virtual environment, the .env secrets file, and Python bytecode caches from source control:

```
venv/
.env
__pycache__/
```

Every development checkpoint followed the same discipline: run `git status` and manually verify no secret or environment file appeared in the changeset before staging and committing.

4. Implementation — Stage by Stage

Stage 1 — Conversational Loop

A minimal loop was built that accepts terminal input, sends it to the Gemini API, and prints the response, repeating until the user exits. Because LLM APIs are stateless between requests, conversational memory was implemented by relying on the Gemini SDK's built-in chat-session object, which resends the accumulated history on each call internally.

A model-naming issue surfaced at this stage: the initially selected model, `gemini-2.5-flash`, returned a 404 error, as it had been retired for new users in favor of newer model identifiers. This was resolved by querying Google's current model listing and switching to a supported model. This highlighted that hosted LLM APIs are a moving target and that model identifiers cannot be treated as permanent.

Stage 2 — File Reading via Tool Calling

A `read_file` tool was implemented and exposed to the model via a JSON schema describing its name, purpose, and parameters. This introduced the project's core mechanism: the model requests a function call, the calling program executes the corresponding local function, and the result is returned to the model as a new message before it produces a final answer.

```
def read_file(path: str) -> str:
    try:
        with open(path, "r") as f:
            return f.read()
    except FileNotFoundError:
        return f"Error: file '{path}' not found."
    except Exception as e:
        return f"Error reading file: {e}"
```

Network resilience was added at this stage via a retry helper implementing exponential backoff for transient server errors (HTTP 5xx), distinguishing them from client errors (HTTP 4xx, such as daily quota exhaustion) which are not retried, since retrying an identical request against an exhausted quota cannot succeed.

```
def send_message_with_retry(message, max_retries=3):
    for attempt in range(1, max_retries + 1):
        try:
            return chat.send_message(message)
        except errors.ServerError:
            time.sleep(2 * attempt)
        except errors.ClientError as e:
            if "RESOURCE_EXHAUSTED" in str(e):
                raise RuntimeError("Daily free-tier quota exceeded.")
            raise
    raise RuntimeError("Gemini API unavailable after retries.")
```

Stage 3 — Safe Text-Replacement Editing

Three designs for representing a proposed edit were evaluated: a full-file rewrite, an exact search-and-replace of a text snippet, and a unified diff patch. The search-and-replace design was

selected, on the basis that it permits a concrete pre-application safety check (the target text must exist exactly once), is trivially convertible to a human-readable diff for review, and avoids the fragility of asking a model to generate a syntactically valid diff patch.

The resulting `edit_file` function verifies that the proposed old text occurs exactly once in the target file, refusing the edit as ambiguous if it occurs more than once, and reporting a clear error if it is absent. Only after this check passes is a diff preview generated (via Python's `difflib.unified_diff`) and shown to the user, followed by an explicit confirmation prompt before any write to disk occurs.

During testing, a defect was identified in the tool-call handling loop: the original implementation assumed at most one function call per user turn, but a single request (e.g. an edit) commonly requires the model to call `read_file` first and `edit_file` second. The handling logic was corrected into a loop that continues executing function calls until the model returns a plain-text response, correctly modelling the fact that a single user request may require an arbitrary number of chained tool calls.

A second defect was found in the rejection path: when a user declined a proposed edit, the initial tool response (“Edit cancelled by user.”) was not directive enough, causing the model to immediately retry the identical edit rather than treating the rejection as final. This was resolved by returning an explicit instruction not to retry the same edit, which materially changed model behavior in subsequent testing.

Stage 4 — File Creation, Deletion, and Modularization

Prior to adding further tools, the codebase was refactored from a single file into the three-module architecture described in Section 2, in order to isolate the increasing complexity of tool implementations from the API and control-flow logic. This refactor was verified to be behavior-preserving by re-running the Stage 3 test suite against the restructured code.

Two further tools were then added, each with a risk-proportionate safety check:

create_file — Refuses outright, without prompting, if the target path already exists, directing the caller toward edit_file instead; otherwise previews the new content and requires a standard yes/no confirmation.

delete_file — Treated as the highest-risk operation in the system, since deletion is not reversible by the tool itself. Rather than a simple yes/no prompt, the user must type the exact filename to confirm, a deliberately higher-friction confirmation modeled on similar patterns in production tools (e.g. repository-deletion confirmations that require typing the repository name).

Stage 5 — Hardening the Confirmation Layer

A final pass addressed edge cases in user input that the earlier implementation handled inelegantly, though not unsafely:

Ambiguous confirmation input: A shared ask_yes_no() helper was introduced that loops until it receives an unambiguous yes/no answer, rather than silently treating any unrecognized input as a rejection.

Interrupt handling mid-prompt: A KeyboardInterrupt (Ctrl+C) raised while the program is waiting at a confirmation prompt is now caught and treated as a cancellation, rather than crashing the process with

an unhandled traceback — a property considered important given that an unexpected crash mid-confirmation leaves the user uncertain whether a file was modified.

Interrupt handling at the main prompt: A KeyboardInterrupt at the top-level input prompt now produces a clean exit message rather than a traceback.

Empty input: Submitting an empty line at the main prompt is now silently ignored rather than being sent to the API as a wasted request.

5. Safety Design: The Confirmation Layer

The project's central design principle is that the model may only *propose* file operations; the program itself decides, on the user's explicit behalf, whether any such operation is actually carried out. This section summarizes the safety mechanism applied to each operation, ordered by increasing risk.

Operation	Pre-check	Confirmation required
read_file	None (read-only, non-destructive).	None.
edit_file	Target text must exist in the file exactly once; refused if not found or ambiguous.	Diff preview shown; standard yes/no confirmation.
create_file	Refused outright if the target path already exists.	Content preview shown; standard yes/no confirmation.
delete_file	File must exist.	User must type the exact filename to confirm; a plain ‘yes’ is not accepte

A further property, verified during testing, is that tool results returned to the model after a rejection must be explicit and directive. An ambiguous result (e.g. simply “cancelled”) was observed to cause the model to retry an identical, already-rejected action; rephrasing the result to explicitly instruct the model not to retry corrected this behavior. This indicates that, in tool-calling systems, the content of a tool's return value functions as a steering instruction for the model's subsequent behavior, not merely as inert status data.

6. Issues Encountered and Resolutions

Issue	Root Cause	Resolution
ModuleNotFoundError for an installed package	A conda environment was active alongside the project venv, so the package was not found	Verified the active interpreter path and ensured the project venv was activated before running
HTTP 404 on a specified model name	The model had been retired for new users and removed from the provider's free-tier model list	Queried the provider's current model list and switched to a supported
HTTP 503 (server overloaded)	Transient overload on the provider's infrastructure, unrelated to the request itself	Implemented retry logic with exponential backoff for this error class.
Tool response silently returned nothing useful	The handling logic assumed at most one function call per turn, so additional calls in a multi-call response were left unh	Restructured the tool-call handling into a loop that processes every call in the turn
HTTP 429 (daily quota exhausted)	The free-tier daily request quota for the selected model was exhausted	Distinguished quota exhaustion (ClientError) from transient errors (ServerE
Rejected edit was retried automatically	The tool result returned on rejection was not forceful enough to prevent the model from re-proposing the same change	Reworded the rejection message to explicitly instruct the model not to

None of the issues encountered stemmed from an incorrect understanding of core programming concepts; nearly all arose from the practical realities of integrating with a live, evolving, externally rate-limited API, and from under-specified assumptions in the tool-calling control flow (single-call vs. multi-call turns; ambiguous vs. directive tool results).

7. Testing Summary

Each safety-relevant code path was manually exercised for both its accept and reject outcomes, rather than only its expected happy path.

Test	Result
edit_file: accept a valid, unambiguous change	Diff shown; change written to disk; verified with a follow-up read.
edit_file: reject a proposed change	File left unmodified on disk; model did not retry the same edit.
edit_file: ambiguous match (text occurs more than once)	Refused before any diff or confirmation prompt; model self-corrected by supply
create_file: create a new file	Content preview shown; file created and verified on disk.
create_file: target path already exists	Refused outright (in this case avoided by the model itself falling back to edit_fil
delete_file: reject via incorrect filename input	File left intact on disk; deletion not performed.
delete_file: confirm via exact filename input	File removed from disk, verified via directory listing.
delete_file: interrupt (Ctrl+C) during confirmation	Deletion cancelled cleanly; file remained on disk; no traceback.
Ambiguous input (e.g. ‘maybe’) at a yes/no prompt	Prompt re-asked until an unambiguous answer was given.
Interrupt (Ctrl+C) at a yes/no prompt	Treated as rejection; handled gracefully with no traceback.
Interrupt (Ctrl+C) at the main input prompt	Program exited cleanly with a farewell message.
Empty input at the main prompt	Silently ignored; no API request issued.

All listed tests passed on the final codebase, and the corresponding changes were committed to version control at each stage.

8. Final Codebase

8.1 tools.py

```
import os
import difflib


def ask_yes_no(prompt: str) -> bool:
    """Ask a y/n question, looping until a clear answer is given.
    Returns True for yes, False for no. Treats Ctrl+C as 'no'."""
    while True:
        try:
            answer = input(prompt).strip().lower()
        except KeyboardInterrupt:
            print("\n[Cancelled]")
            return False
        if answer in ("y", "yes"):
            return True
        if answer in ("n", "no"):
            return False
        print("Please answer 'y' or 'n'.")


def read_file(path: str) -> str:
    try:
        with open(path, "r") as f:
            return f.read()
    except FileNotFoundError:
        return f"Error: file '{path}' not found."
    except Exception as e:
        return f"Error reading file: {e}"


def edit_file(path: str, old_text: str, new_text: str) -> str:
    try:
        with open(path, "r") as f:
            content = f.read()
    except FileNotFoundError:
        return f"Error: file '{path}' not found."
    except Exception as e:
        return f"Error reading file: {e}"

    count = content.count(old_text)
    if count == 0:
        return (f"Error: could not find the exact text to replace in '{path}'. "
                f"Make sure old_text matches the file exactly, including whitespace.")
    if count > 1:
        return (f"Error: the text to replace appears {count} times in '{path}', "
                f"which is ambiguous. Provide more surrounding context to make it uniqu
e.")

    new_content = content.replace(old_text, new_text)

    diff = difflib.unified_diff(
        content.splitlines(keepends=True),
        new_content.splitlines(keepends=True),
```

```
        fromfile=f"{path} (before)", tofile=f"{path} (after)"
    )
    print("\n--- Proposed change ---")
    print("".join(diff))
    print("-------------------------")

    if not ask_yes_no("Apply this change? (y/n): "):
        return ("The user REJECTED this edit and does not want it applied. "
                "Do not attempt this same edit again.")

    with open(path, "w") as f:
        f.write(new_content)
    return f"Successfully edited '{path}'."


def create_file(path: str, content: str) -> str:
    if os.path.exists(path):
        return f"Error: '{path}' already exists. Use edit_file instead."

    print(f"\n--- Proposed new file: {path} ---")
    print(content)
    print("-------------------------")

    if not ask_yes_no(f"Create '{path}'? (y/n): "):
        return "The user REJECTED creating this file. Do not attempt again."

    try:
        with open(path, "w") as f:
            f.write(content)
    except Exception as e:
        return f"Error creating file: {e}"
    return f"Successfully created '{path}'."


def delete_file(path: str) -> str:
    if not os.path.exists(path):
        return f"Error: '{path}' does not exist, nothing to delete."

    print(f"\n!!! DESTRUCTIVE ACTION: this will permanently delete '{path}' !!!")
    try:
        confirm = input(f"Type the filename exactly to confirm deletion of '{path}': ")
.strip()
    except KeyboardInterrupt:
        print("\n[Cancelled]")
        return "The user cancelled the deletion (Ctrl+C). Do not retry."

    if confirm != path:
        return "The user did NOT confirm deletion. Do not attempt to delete again."

    try:
        os.remove(path)
    except Exception as e:
        return f"Error deleting file: {e}"
    return f"Successfully deleted '{path}'."
```

```
read_file_tool = {"name": "read_file", "description": "Read a file's contents.",
    "parameters": {"type": "object",
        "properties": {"path": {"type": "string"}}, "required": ["path"]}}

edit_file_tool = {"name": "edit_file",
    "description": "Replace an exact snippet of text in a file with new text.",
    "parameters": {"type": "object",
        "properties": {"path": {"type": "string"}, "old_text": {"type": "string"},
                       "new_text": {"type": "string"}},
        "required": ["path", "old_text", "new_text"]}}

create_file_tool = {"name": "create_file",
    "description": "Create a new file with given content. Fails if it exists.",
    "parameters": {"type": "object",
        "properties": {"path": {"type": "string"}, "content": {"type": "string"}},
        "required": ["path", "content"]}}

delete_file_tool = {"name": "delete_file",
    "description": "Permanently delete a file. Irreversible.",
    "parameters": {"type": "object",
        "properties": {"path": {"type": "string"}}, "required": ["path"]}}

TOOL_FUNCTIONS = {
    "read_file": read_file, "edit_file": edit_file,
    "create_file": create_file, "delete_file": delete_file,
}
ALL_TOOL_SCHEMAS = [read_file_tool, edit_file_tool, create_file_tool, delete_file_tool]
```

8.2 api.py

```
import time
from dotenv import load_dotenv
load_dotenv()

from google import genai
from google.genai import types
from google.genai import errors
from tools import ALL_TOOL_SCHEMAS

client = genai.Client()
tools = types.Tool(function_declarations=ALL_TOOL_SCHEMAS)
config = types.GenerateContentConfig(tools=[tools])
chat = client.chats.create(model="gemini-3.1-flash-lite", config=config)


def send_message_with_retry(message, max_retries=3):
    for attempt in range(1, max_retries + 1):
        try:
            return chat.send_message(message)
        except errors.ServerError:
            print(f"\n[Server busy, retrying... attempt {attempt}/{max_retries}]")
            time.sleep(2 * attempt)
        except errors.ClientError as e:
            if "RESOURCE_EXHAUSTED" in str(e) or "429" in str(e):
                raise RuntimeError("Daily free-tier quota exceeded for this model.")
            raise
```

```
    raise RuntimeError("Gemini API unavailable after several retries.")
```

8.3 assistant.py

```
from google.genai import types
from api import send_message_with_retry
from tools import TOOL_FUNCTIONS


def main():
    print("CLI Assistant (type 'exit' to quit)")
    while True:
        try:
            user_input = input("\nYou: ")
        except KeyboardInterrupt:
            print("\nGoodbye!")
            break

        if not user_input.strip():
            continue
        if user_input.strip().lower() == "exit":
            break

        try:
            response = send_message_with_retry(user_input)
        except RuntimeError as e:
            print(f"\n[Error: {e}]")
            continue

        while True:
            function_call = None
            for part in response.candidates[0].content.parts:
                if part.function_call:
                    function_call = part.function_call
                    break
            if not function_call:
                break

            print(f"\n[Assistant wants to call: "
                  f"{function_call.name}({dict(function_call.args)})]")

            func = TOOL_FUNCTIONS.get(function_call.name)
            result = func(**function_call.args) if func else \
                f"Error: unknown function {function_call.name}"

            function_response = types.Part.from_function_response(
                name=function_call.name, response={"result": result})
            try:
                response = send_message_with_retry(function_response)
            except RuntimeError as e:
                print(f"\n[Error: {e}]")
                break

        print(f"\nAssistant: {response.text}")
```

```
if __name__ == "__main__":
    main()
```

9. Conclusion and Future Work

The project successfully implements all five originally scoped capabilities: a persistent conversational loop, file reading, safe precision-based file editing, file creation and deletion, and a confirmation layer that was iteratively hardened against ambiguous input, interrupts, and model retry behavior after rejection. Every safety-critical path was tested against both its intended and its adversarial cases, rather than validated only on a single happy-path scenario.

The most significant technical lesson from the project concerns the nature of tool-calling systems: the model does not execute actions, it only requests them, and the calling program's tool results function as active steering signals for the model's subsequent behavior rather than passive status reports. Both of the defects found during testing (unhandled multi-call turns, and retried edits after rejection) stemmed from under-specifying this relationship, and both were resolved by making the control flow and the tool-result messages more explicit.

Potential directions for future work include:

- A list_files / directory-listing tool, so the model does not need to guess file paths from context alone.
- A session-level 'trust mode' allowing confirmations to be temporarily bypassed for a batch of low-risk operations, with a clear warning banner.
- Persisting conversation history to disk, so a session can be resumed after the program exits.
- Support for an additional LLM provider as an automatic fallback when the primary provider's free-tier quota is exhausted.

Report prepared by Shaurya Pathak.